

Отчёт по лабораторной работе №9

Дисциплина: Архитектура компьютеров

Мухина Ксения Николаевна

Содержание

1	Цель работы	5
2	Выполнение лабораторной работы	6
3	Задания для самостоятельной работы	19
4	Выводы	24
	Список литературы	25

Список иллюстраций

2.1	Создание каталога и файла для работы.	6
2.2	Программа lab09-1.	6
2.3	Дополнение программы lab09-1.	7
2.4	Изменённая программа lab09-1.	7
2.5	Программа lab09-2.	8
2.6	Установка брейкпоинта.	8
2.7	Дизассемблированный код lab09-2 с синтаксисом АТТ.	9
2.8	Дизассемблированный код lab09-2 с синтаксисом Intel.	10
2.9	Интерфейс отладчика GDB.	11
2.10	Проверка наличия брейкпоинта.	12
2.11	Установка брейкпоинта по адресу.	12
2.12	Регистры до выполнения программы.	13
2.13	Регистры после выполнения 5 инструкций.	13
2.14	Значения переменных msg1 и msg2.	14
2.15	Изменение значений msg1 и msg2.	14
2.16	Значение регистра edx в разных форматах.	15
2.17	Изменение значения регистра ebx.	16
2.18	Завершение программы lab09-2.	16
2.19	Подготовка к анализу программы lab09-3.	17
2.20	Адрес вершины стека в регистре esp.	17
2.21	Адреса позиций стека.	18
3.1	Программа task1.	21
3.2	Программа task2.	21
3.3	Значения регистров при выполнении некоторых инструкций.	22
3.4	Изменение программы task2.	23
3.5	Программа task2.	23

Список таблиц

1 Цель работы

Приобрести навыки написания программ с использованием подпрограмм. Ознакомиться с методами отладки при помощи GDB и его основными возможностями.

2 Выполнение лабораторной работы

Далее описываемая работа была выполнена на виртуальной машине Oracle VirtualBox с ОС Linux Ubuntu.

1. Реализация подпрограмм в NASM

Перед началом работы создадим каталог lab09 и файл lab09-1.asm.

```
knmuhina@virtualbox:~$ mkdir ~/work/arch-pc/lab09
knmuhina@virtualbox:~$ cd ~/work/arch-pc/lab09
knmuhina@virtualbox:~/work/arch-pc/lab09$ touch lab09-1.asm
```

Рисунок 2.1: Создание каталога и файла для работы.

Рассмотрим в качестве примера программу вычисления выражения „ $f(x) = 2x + 7$ “ с помощью подпрограммы `_calcul`. Значение x вводится с клавиатуры, а процесс вычисления проходит в подпрограмме. Введём в файл текст программы из листинга, представленного в лабораторной работе, и создадим исполняемый файл.

```
knmuhina@virtualbox:~/work/arch-pc/lab09$ nasm -f elf lab09-1.asm
knmuhina@virtualbox:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab09-1 lab09-1.o
knmuhina@virtualbox:~/work/arch-pc/lab09$ ./lab09-1
Введите x: 5
2x + 7 = 17
knmuhina@virtualbox:~/work/arch-pc/lab09$
```

Рисунок 2.2: Программа lab09-1.

В данной программе функция `'call _calcul'` вызывает данную подпрограмму, выполняет инструкции внутри неё, затем при помощи `ret` выходит из подпро-

граммы и продолжает выполнение с инструкции, следующей за call. Остальные подпрограммы, извлекаемые из файла in_out.asm, работают таким же образом. Предназначения данных подпрограмм уже известны.

Теперь изменим текст программы, добавив в подпрограмму _calcul другую подпрограмму - _subcalcul, где x становится переменной в выражении „ $g(x) = 3x - 1$ “, а результат $g(x)$ - переменной f(x).

```

; -----
; Подпрограмма вычисления
; выражения "2x + 7", x = g(x)
; -----

; -----
; Подпрограмма вычисления
; выражения "2x + 7"
; -----

_calcul:
mov ebx, 2
mul ebx
add eax, 7
mov [res], eax
ret ; выход из подпрограммы

; -----
; Подпрограмма вычисления
; выражения "3x - 1"
; -----

_subcalcul:
mov ebx, 3
mul ebx
sub eax, 1
ret
  
```

Рисунок 2.3: Дополнение программы lab09-1.

Создадим исполняемый файл и запустим его.

```

knmuhina@virtualbox:~/work/arch-pc/lab09$ nasm -f elf lab09-1.asm
knmuhina@virtualbox:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab09-1 lab09-1.o
knmuhina@virtualbox:~/work/arch-pc/lab09$ ./lab09-1
Введите x: 5
2x + 7 = 35
knmuhina@virtualbox:~/work/arch-pc/lab09$
  
```

Рисунок 2.4: Изменённая программа lab09-1.

Результат программы корректный.

2. Отладка программ с помощью GDB

Создадим файл lab09-2.asm, введём в файл текст программы из листинга, представленного в лабораторной работе, и создадим исполняемый файл. Для

работы с GDB в исполняемый файл необходимо добавить отладочную информацию, поэтому проведём трансляцию с ключом „g“. Сам исполняемый файл загрузим в отладчик GDB.

```
knmuhina@virtualbox:~/work/arch-pc/lab09$ nasm -f elf -g -l lab09-2.lst lab09-2.asm
knmuhina@virtualbox:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab09-2 lab09-2.o
knmuhina@virtualbox:~/work/arch-pc/lab09$ gdb lab09-2
GNU gdb (Ubuntu 16.3-1ubuntu2) 16.3
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from lab09-2...
(gdb) run
Starting program: /home/knmuhina/work/arch-pc/lab09/lab09-2

Hello, world!
[Inferior 1 (process 10559) exited normally]
```

Рисунок 2.5: Программа lab09-2.

Программа работает корректно, выводя сообщение „Hello, world!“.

Для более подробного анализа программы установим брейкпоинт на метку `_start` и запустим программу.

```
(gdb) break _start
Breakpoint 1 at 0x00490000: file lab09-2.asm, line 12.
(gdb) run
Starting program: /home/knmuhina/work/arch-pc/lab09/lab09-2

Breakpoint 1, _start () at lab09-2.asm:12
12      mov eax, 4
```

Рисунок 2.6: Установка брейкпоинта.

Посмотрим дизассемблированный код программы при помощи `disassemble`, начиная с метки `_start`.


```

(gdb) disassemble _start
Dump of assembler code for function _start:
=> 0x08049000 <+0>:      mov     $0x4,%eax
    0x08049005 <+5>:      mov     $0x1,%ebx
    0x0804900a <+10>:     mov     $0x804a000,%ecx
    0x0804900f <+15>:     mov     $0x8,%edx
    0x08049014 <+20>:     int     $0x80
    0x08049016 <+22>:     mov     $0x4,%eax
    0x0804901b <+27>:     mov     $0x1,%ebx
    0x08049020 <+32>:     mov     $0x804a008,%ecx
    0x08049025 <+37>:     mov     $0x7,%edx
    0x0804902a <+42>:     int     $0x80
    0x0804902c <+44>:     mov     $0x1,%eax
    0x08049031 <+49>:     mov     $0x0,%ebx
    0x08049036 <+54>:     int     $0x80
End of assembler dump.

```

Рисунок 2.7: Дизассемблированный код lab09-2 с синтаксисом АТТ.

В данном случае команды отражаются с синтаксисом АТТ.

Переключим синтаксис отображения команд на Intel при помощи команды „set disassembly-flavor Intel“ и снова посмотрим код программы.

```

(gdb) set disassembly-flavor intel
(gdb) disassemble _start
Dump of assembler code for function _start:
=> 0x08049000 <+0>:      mov     eax,0x4
    0x08049005 <+5>:      mov     ebx,0x1
    0x0804900a <+10>:     mov     ecx,0x804a000
    0x0804900f <+15>:     mov     edx,0x8
    0x08049014 <+20>:     int     0x80
    0x08049016 <+22>:     mov     eax,0x4
    0x0804901b <+27>:     mov     ebx,0x1
    0x08049020 <+32>:     mov     ecx,0x804a008
    0x08049025 <+37>:     mov     edx,0x7
    0x0804902a <+42>:     int     0x80
    0x0804902c <+44>:     mov     eax,0x1
    0x08049031 <+49>:     mov     ebx,0x0
    0x08049036 <+54>:     int     0x80
End of assembler dump.
(gdb)

```

Рисунок 2.8: Дизассемблированный код lab09-2 с синтаксисом Intel.

Несмотря на одинаковый результат, синтаксисы имеют некоторую разницу в отображении. В отличие от кода в синтаксисе АТТ, в Intel-синтаксисе операнды идут в порядке назначения-источник, а не наоборот. Также сами операнды записываются без префиксов.

Включим режим псевдографики для более удобного анализа программы при помощи „layout asm“ и „layout regs“. Теперь интерфейс будет выглядеть следующим образом:

```
Register group: general
eax      0x0      0
ecx      0x0      0
edx      0x0      0
ebx      0x0      0
esp      0xffffd020 0xffffd020
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0
eip      0x8049000 0x8049000 <_start>
eflags   0x202    [ IF ]
cs       0x23     35
ss       0x2b     43
ds       0x2b     43
es       0x2b     43
fs       0x0      0
gs       0x0      0

B+>0x8049000 <_start> mov    eax,0x4
0x8049005 <_start+5> mov    ebx,0x1
0x804900a <_start+10> mov    ecx,0x804a000
0x804900f <_start+15> mov    edx,0x8
0x8049014 <_start+20> int    0x80
0x8049016 <_start+22> mov    eax,0x4
0x804901b <_start+27> mov    ebx,0x1
0x8049020 <_start+32> mov    ecx,0x804a008
0x8049025 <_start+37> mov    edx,0x7
0x804902a <_start+42> int    0x80
0x804902c <_start+44> mov    eax,0x1
0x8049031 <_start+49> mov    ebx,0x0
0x8049036 <_start+54> int    0x80
0x8049038 add    BYTE PTR [eax],al
0x804903a add    BYTE PTR [eax],al
0x804903c add    BYTE PTR [eax],al
0x804903e add    BYTE PTR [eax],al

native process 10567 (asm) In: _start L12 PC: 0x8049000
(gdb) layout regs
(gdb)
```

Рисунок 2.9: Интерфейс отладчика GDB.

В этом режиме верхняя часть окна выделена под отображение информации о регистрах, средняя - отображение дизассемблированного кода, а нижняя часть доступная для ввода команд.

2.1. Добавление точек останова

Установить точку останова (брейкпоинт) можно командой `break` (или `b`). Типичный аргумент команды - место установки. Его можно задать как номер строки программы, имя метки или адрес. Перед адресом ставится знак «звёздочки» для избежания путаницы с номерами.

На одном из предыдущих шагов был установлен брейкпоинт по имени метки. Проверим это при помощи „`info breakpoints`“ (или „`i b`“)

```
(gdb) i b
Num      Type           Disp Enb Address      What
1        breakpoint     keep y  0x08049000 lab09-2.asm:12
        breakpoint already hit 1 time
```

Рисунок 2.10: Проверка наличия брейкпоинта.

Установим ещё один брейкпоинт по адресу инструкции, который можно увидеть в средней части экрана в левом столбце инструкции. Определим адрес инструкции „mov ebx, 0x0“ и установим брейкпоинт. Затем снова посмотрим информацию о брейкпоинтах.

```
(gdb) break *0x8049031
Breakpoint 2 at 0x8049031: file lab09-2.asm, line 25.
(gdb) i b
Num      Type           Disp Enb Address      What
1        breakpoint     keep y  0x08049000 lab09-2.asm:12
        breakpoint already hit 1 time
2        breakpoint     keep y  0x08049031 lab09-2.asm:25
(gdb)
```

Рисунок 2.11: Установка брейкпоинта по адресу.

Теперь установлены два брейкпоинта - по метке `_start` и по адресу `0x8049031`.

2.2. Работа с данными программы в GDB

Отладчик может показывать содержимое ячеек памяти и регистров, а при необходимости позволяет вручную изменить значения регистров и переменных.

Посмотрим содержимое регистров через команду „info registers“ (или „i r“).

```

(gdb) i r
eax          0x0          0
ecx          0x0          0
edx          0x0          0
ebx          0x0          0
esp          0xffffd020    0xffffd020
ebp          0x0          0x0
esi          0x0          0
edi          0x0          0
eip          0x8049000    0x8049000 <_start>
eflags       0x202        [ IF ]
cs           0x23         35
ss           0x2b         43
ds           0x2b         43
es           0x2b         43
fs           0x0          0
gs           0x0          0
(gdb)

```

Рисунок 2.12: Регистры до выполнения программы.

Выполним 5 инструкций с помощью команды „stepi 5“ (или „si 5“) и проверим значения регистров на наличие изменений.

```

eax          0x8          8
ecx          0x804a000    134520832
edx          0x8          8
ebx          0x1          1
esp          0xffffd020    0xffffd020
ebp          0x0          0x0
esi          0x0          0
edi          0x0          0
eip          0x8049016    0x8049016 <_start+22>
eflags       0x202        [ IF ]
cs           0x23         35
ss           0x2b         43
ds           0x2b         43
es           0x2b         43
fs           0x0          0
gs           0x0          0

```

Рисунок 2.13: Регистры после выполнения 5 инструкций.

Как мы видим, изменились значения регистров `eax`, `ebx`, `ecx`, `edx` и `eip`.

Для отображения содержимого памяти можно использовать команду „x“, которая выдаёт содержимое ячейки по указанному адресу. Формат вывода

данных можно задать после имени команды через косую черту: „x/NFU “.

С помощью команды „x &“ можно посмотреть содержимое переменной.

Посмотрим значение переменной msg1 по имени, а переменной msg2 - по адресу. Адрес msg2 определим по дизассемблированной инструкции „mov esx, msg2“.

```
(gdb) x/1sb &msg1
0x804a000 <msg1>:      "Hello, "
(gdb) x/1sb 0x804a008
0x804a008 <msg2>:      "world!\n\034"
(gdb)
```

Рисунок 2.14: Значения переменных msg1 и msg2.

С помощью команды set можно изменить значение для регистра или ячейки памяти, задав в качестве аргумента имя регистра или адреса. Для имени регистра команда будет выглядеть как „set \$“, для адреса - как „set {type}“. В обоих случаях обязательно указываются префикс \$ и тип данных соответственно.

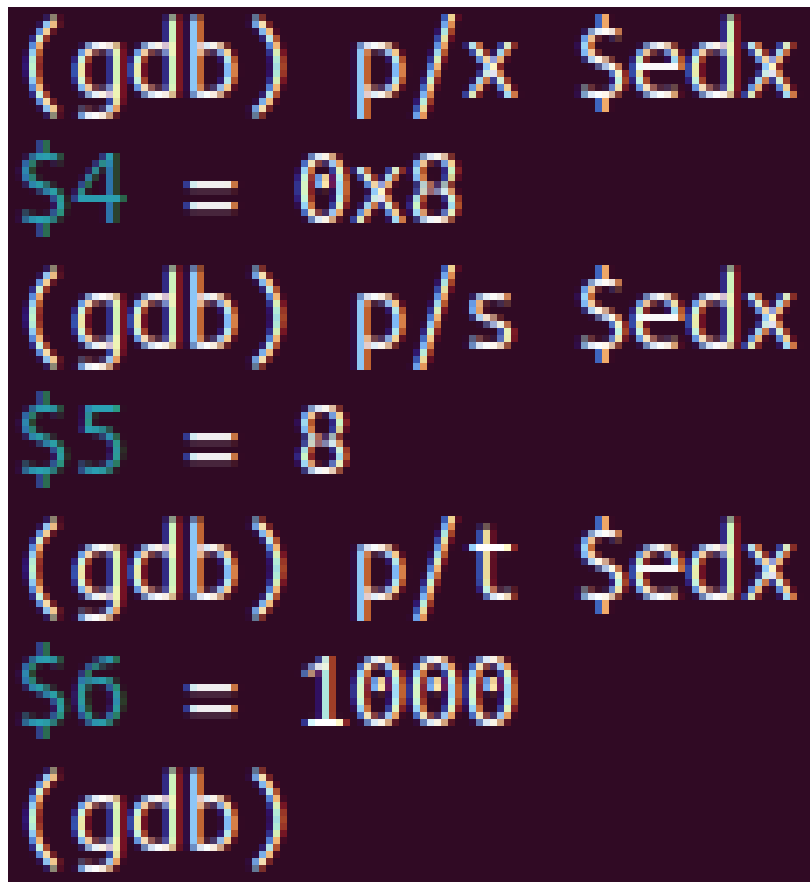
Изменим первые символы в переменных msg1 и msg2, задав как аргументы их адреса.

```
(gdb) set {char}0x804a000='h'
(gdb) x/1sb &msg1
0x804a000 <msg1>:      "hello, "
(gdb) set {char}0x804a008='W'
(gdb) x/1sb &msg2
0x804a008 <msg2>:      "World!\n\034"
(gdb)
```

Рисунок 2.15: Изменение значений msg1 и msg2.

Для просмотра значений регистров используется команда „print /F “ (или „p/F “). Перед именем регистра также ставится префикс \$.

Выведем значение регистра `edx` в шестнадцатеричном, в двоичном форматах и в символьном виде.



```
(gdb) p/x $edx
$4 = 0x8
(gdb) p/s $edx
$5 = 8
(gdb) p/t $edx
$6 = 1000
(gdb)
```

Рисунок 2.16: Значение регистра `edx` в разных форматах.

С помощью `set` изменим значение регистра `ebx`.

```
(gdb) set $ebx='2'  
(gdb) p/s $ebx  
$7 = 50  
(gdb) set $ebx=2  
(gdb) p/s $ebx  
$8 = 2  
(gdb)
```

Рисунок 2.17: Изменение значения регистра ebx.

Разница вывода обуславливается тем, что сначала мы меняем значение регистра на символ „2“, код которого выводится при применении p/s. Затем мы меняем значение на число 2, которое и отображается при выводе команды p/s.

Завершим выполнение программы при помощи continue (или c) и выйдем из GDB.

```
(gdb) c  
Continuing.  
World!  
Breakpoint 2, _start () at lab09-2.asm:25  
(gdb) c  
Continuing.  
[Inferior 1 (process 10967) exited normally]  
(gdb) quit
```

Рисунок 2.18: Завершение программы lab09-2.

2.3. Обработка аргументов командной строки в GDB

Скопируем файл lab8-2.asm, созданный в ходе выполнения лабораторной работы №8 в файл с именем lab09-3.asm. Создадим исполняемый файл и загрузим его в gdb, задав аргументы „аргумент1“, „аргумент“, „2“ и „аргумент 3“ с использованием ключа `–args`.

Также перед началом анализа программы установим брейкпоинт в метке `_start` и запустим её.

```
knmuhina@virtualbox:~/work/arch-pc/lab09$ cp ~/work/arch-pc/lab08/lab8-2.asm ~/work/arch-pc/lab09/lab09-3.asm
knmuhina@virtualbox:~/work/arch-pc/lab09$ nasm -f elf -g -l lab09-3.lst lab09-3.asm
knmuhina@virtualbox:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab09-3 lab09-3.o
knmuhina@virtualbox:~/work/arch-pc/lab09$ gdb --args lab09-3 аргумент1 аргумент 2 'аргумент 3'
GNU gdb (Ubuntu 16.3-1ubuntu2) 16.3
Copyright (C) 2016 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from lab09-3...
(gdb) b _start
Breakpoint 1 at 0x00490e8: file lab09-3.asm, line 6.
(gdb) run
Starting program: /home/knmuhina/work/arch-pc/lab09/lab09-3 аргумент1 аргумент 2 аргумент\ 3

Breakpoint 1, _start () at lab09-3.asm:6
6      pop ecx
```

Рисунок 2.19: Подготовка к анализу программы lab09-3.

Как отмечалось в предыдущей работе, при запуске программы заданные аргументы загружаются в стек. Исследуем расположение аргументов в стеке.

Адрес вершины стека хранится в регистре `esp`, и по этому адресу располагается число, равное количеству аргументов (включая имя программы). Посмотрим его, используя команду „`x/x`“.

```
(gdb) x/x $esp
0xffffd070:      0x00000005
(gdb)
```

Рисунок 2.20: Адрес вершины стека в регистре `esp`.

Мы убедились, что число аргументов равно 5: в него входят имя программы lab09-3 и непосредственно 4 заданных аргумента.

По адресу [esp+4] располагается адрес имени программы, по адресу [esp+8] - адрес первого аргумента, и т.д. Посмотрим остальные позиции стека через команду 'x/s *(void**)(%)'.

```
(gdb) x/s *(void**)(%esp + 4)
0xfffffd23b:    "/home/knmuhina/work/arch-pc/lab09/lab09-3"
(gdb) x/s *(void**)(%esp + 8)
0xfffffd265:    "аргумент1"
(gdb) x/s *(void**)(%esp + 12)
0xfffffd277:    "аргумент"
(gdb) x/s *(void**)(%esp + 16)
0xfffffd288:    "2"
(gdb) x/s *(void**)(%esp + 20)
0xfffffd28a:    "аргумент 3"
(gdb) x/s *(void**)(%esp + 24)
0x0:    <error: Cannot access memory at address 0x0>
(gdb)
```

Рисунок 2.21: Адреса позиций стека.

Шаг изменения адреса равен 4, так как каждый аргумент функции занимает в стеке 4 байта.

3 Задания для самостоятельной работы

Изменим программу, написанную по заданию для самостоятельной работы из лабораторной работы №8, реализовав вычисление значения функции $f(x)$ как подпрограмму.

Листинг 1. Программа task1.asm

```
%include 'in_out.asm'

SECTION .data
msg db 'Функция:  $f(x) = 15x - 9$ ', 0h
msgR db 'Результат: ', 0h

SECTION .text
global _start
_start:
    pop ecx
    pop edx
    sub ecx, 1
    mov esi, 0

next:
```

```
    cmp ecx, 0
    jz _end
    pop eax
    call atoi
    call _calc
    loop next

_end:
    mov eax, msg
    call sprintf
    mov eax, msgR
    call sprintf
    mov eax, esi
    call iprintLF
    call quit

_calc:
    mov ebx, 15
    mul ebx
    sub eax, 9
    add esi, eax

    ret
```

Создадим исполняемый файл и проверим работу программы для набора аргументов.

```

knmuhina@virtualbox:~/work/arch-pc/lab09$ nasm -f elf task1.asm
knmuhina@virtualbox:~/work/arch-pc/lab09$ ld -m elf_i386 -o task1 task1.o
knmuhina@virtualbox:~/work/arch-pc/lab09$ ./task1 1 4 2 3
Функция: f(x) = 15x - 9
Результат: 114
knmuhina@virtualbox:~/work/arch-pc/lab09$

```

Рисунок 3.1: Программа task1.

Программа работает корректно.

В листинге, представленном в задании 2, приведена программа вычисления выражения $(3 + 2) * 4 + 5$. Проверим, что при запуске данная программа даёт неверный результат. Введём в файл task2.asm текст программы из листинга и создадим исполняемый файл.

```

knmuhina@virtualbox:~/work/arch-pc/lab09$ touch task2.asm
knmuhina@virtualbox:~/work/arch-pc/lab09$ mc
knmuhina@virtualbox:~/work/arch-pc/lab09$ nasm -f elf task2.asm
knmuhina@virtualbox:~/work/arch-pc/lab09$ ld -m elf_i386 -o task2 task2.o
knmuhina@virtualbox:~/work/arch-pc/lab09$ ./task2
Результат: 10
knmuhina@virtualbox:~/work/arch-pc/lab09$

```

Рисунок 3.2: Программа task2.

Программа действительно даёт неверный результат.

Проанализируем программу с помощью отладчика GDB. Открыв программу через отладчик и проверив изменения значений регистров, постепенно выполняя инструкции при помощи `si`, мы обнаружили следующее некорректное изменение значений:

eax	0x2	2
ecx	0x4	4
edx	0x0	0
ebx	0x5	5
esp	0xffffd020	0xffffd020
ebp	0x0	0x0
esi	0x0	0
edi	0x0	0



eax	0x8	8
ecx	0x4	4
edx	0x0	0
ebx	0x5	5
esp	0xffffd020	0xffffd020
ebp	0x0	0x0
esi	0x0	0
edi	0x0	0

Рисунок 3.3: Значения регистров при выполнении некоторых инструкций.

При выполнении инструкции „mov ecx, 4“ программа ожидаемо записывает значение 4 в регистр ecx. Однако при выполнении „mul ecx“ значение умножается не на 5 (как оно изначально должно было быть), а на 2 - текущее значение регистра eax. Это связано с тем, что при складывании значений eax и ebx результат записался в ebx и не был перемещён в eax. В итоге значение ebx остаётся неизменным.

Ещё одной ошибкой является вывод на экран результата значения, записанного в регистр ebx, к которому вследствие предыдущей ошибки вычисления было прибавлено 5.

Изменим текст программы таким образом, чтобы программа выводила верный результат.

<pre> ; ---- Вычисление выражения (3 + 2)*4 + 5 mov ebx, 3 mov eax, 2 add ebx, eax mov ecx, 4 mul ecx add ebx, 5 mov edi, ebx ; ---- Вывод результата на экран mov eax, div call sprint mov eax, edi call iprintLF call quit </pre>	→	<pre> ; ---- Вычисление выражения (3 + 2)*4 + 5 mov ebx, 3 mov eax, 2 add ebx, eax mov eax, ebx mov ecx, 4 mul ecx add eax, 5 mov edi, eax ; ---- Вывод результата на экран mov eax, div call sprint mov eax, edi call iprintLF call quit </pre>
--	---	---

Рисунок 3.4: Изменение программы task2.

Создадим исполняемый файл и проверим работу программы.

```

knmuhina@virtualbox:~/work/arch-pc/lab09$ nasm -f elf task2.asm
knmuhina@virtualbox:~/work/arch-pc/lab09$ ld -m elf_i386 -o task2 task2.o
knmuhina@virtualbox:~/work/arch-pc/lab09$ ./task2
Результат: 25
knmuhina@virtualbox:~/work/arch-pc/lab09$

```

Рисунок 3.5: Программа task2.

Теперь программа работает корректно.

4 Выводы

В ходе выполнения лабораторной работы мы приобрели навыки написания программ с использованием подпрограмм и ознакомились с методами отладки при помощи GDB и его основными возможностями.

Список литературы

1. Файл «Лабораторная работа №9. Понятие подпрограммы. Отладчик GDB.pdf»